

Algoritmi e Strutture Dati–parte I

lez. 18

Francesco Camastra

francesco.camastra@uniparthenope.it

Docente

- Prof. Francesco Camastra

Orario lezioni

- Mercoledì 11.00-13.00
(aula 11, Secondo Piano)
- Giovedì 9.00-11.00 (aula 11, Secondo Piano)

Ricevimento

- Orario: Mercoledì 14.00-18.00
- Dove: Stanza 430, Quarto piano, Centro Direzionale, Isola C4
- Per il ricevimento è **OBBLIGATORIO** inviare una email di prenotazione almeno due giorni prima del giorno di ricevimento.
- email del Docente:
francesco.camastra@uniparthenope.it
- Pagina del Docente:
dist.uniparthenope.it/francesco.camastra

Prerequisiti

- Programmazione I
- Matematica I
- Programmazione II
- **NOTA BENE:** Le propedeuticità non sono obbligatorie ... Ossia potreste anche sostenere l'esame senza aver dato Programmazione I e Programmazione II

Frequenza

- La Frequenza del corso non è obbligatoria

Modalità dell' esame

- Prova scritta di Algoritmi e Strutture Dati I ovvero Due prove intercorso (validità quattro appelli)
- Svolgimento di un progetto Algoritmi e Strutture Dati II da scegliersi tra una lista di progetti che verrà comunicata dal docente (validità quattro appelli).
- Superato lo scritto di ASD-I ed il progetto di ASD-II si accede all' Esame Orale congiunto di ASD-I e ASD-II

Algoritmo di Kruskal

1. $A \leftarrow \emptyset$
2. **for** ogni vertice $v \in V[G]$
3. do MAKE-SET(V)
4. Sort(E)
5. // Ordina gli archi di E per peso w non decrescente
6. **for** ogni arco $(u, v) \in E[G]$
7. **if** FIND-SET(u) $\neq$ FIND-SET(v)
8. $A = A \cup \{(u, v)\}$
9. Union(u, v)
10. **return** A

Algoritmo di Prim

L'algoritmo di Prim procede mantenendo in A un singolo albero

- L'albero parte da un vertice arbitrario r (la *radice*) e cresce fino a quando non ricopre tutti i vertici
- Ad ogni passo viene aggiunto un arco leggero che collega un vertice in V_A con un vertice in $V - V_A$, dove V_A è l'insieme di nodi raggiunti da archi in A .

(cont.)

Correttezza

- $(VA, V - VA)$ è un taglio che rispetta A (per definizione)
- Per il corollario, gli archi leggeri che attraversano il taglio sono sicuri

Implementazione

- Durante l'esecuzione, i vertici non ancora nell'albero si trovano in una **coda con priorità** Q ordinata in base ad un campo *key* che è il peso minimo di un arco che collega v ad un vertice nell'albero, o $+\infty$ se tale arco non esiste
- L'insieme $V - Q$ contiene i vertici dell'albero che si sta costruendo.
- Ogni nodo v **mantiene un puntatore al padre** $\pi[v]$

(cont.)

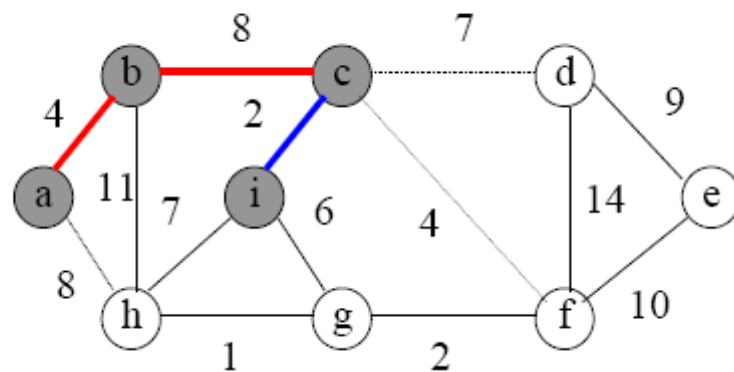
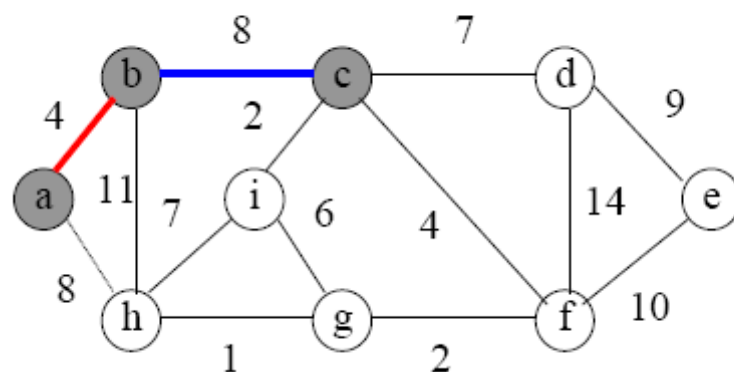
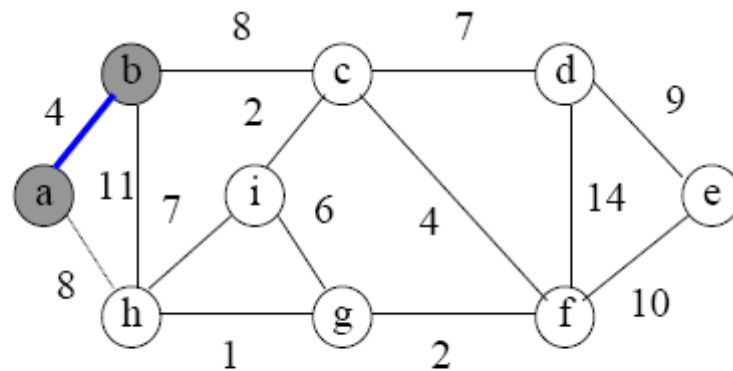
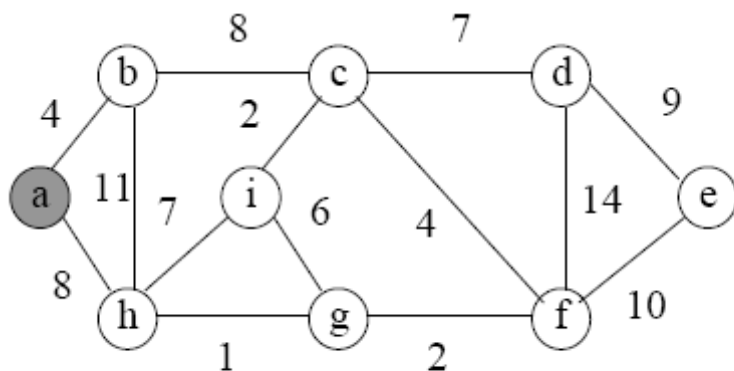
- **Terminazione: Quando l'insieme Q è vuoto**
- Tutti i nodi tranne la radice conoscono il proprio padre

Algoritmo di Prim

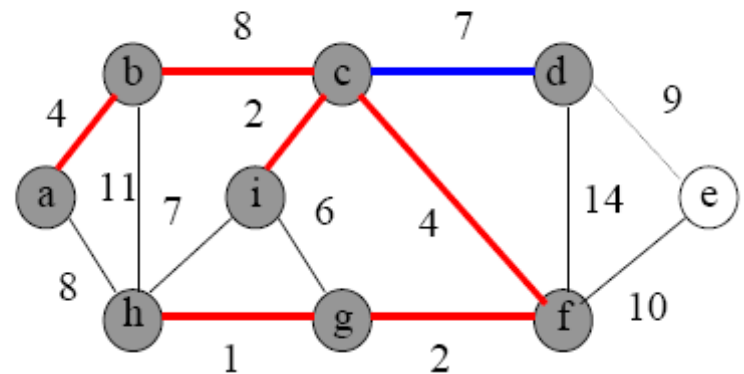
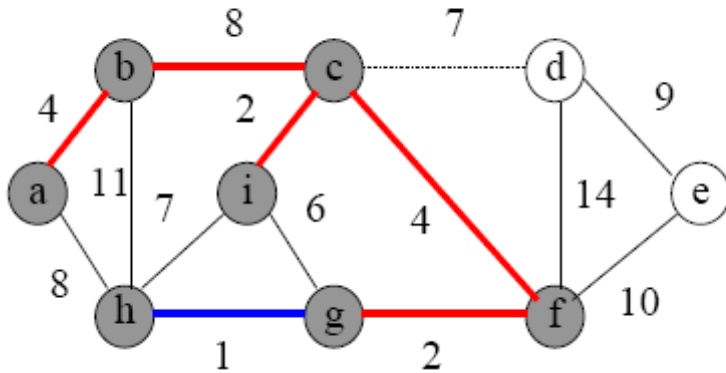
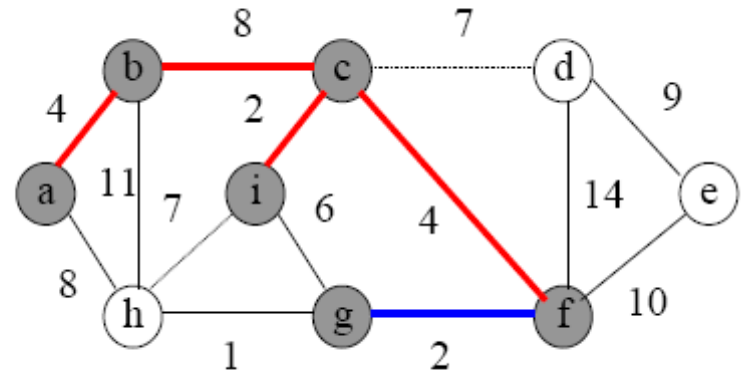
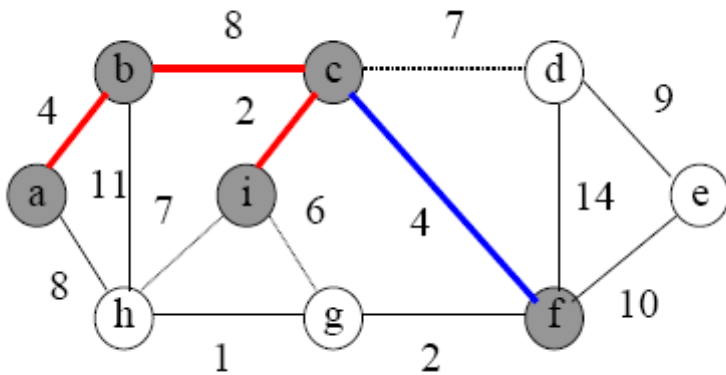
MST_PRIM(G, w, r)

1. $Q \leftarrow V[G]$
2. **for** each u in $V[G] - \{r\}$
3. $key[u] \leftarrow +\infty$
4. $key[r] \leftarrow 0$
5. $\pi[r] \leftarrow NIL$
6. **while** $Q \neq \emptyset$
7. $u \leftarrow extractMin(Q)$
8. **for** each $v \in Adj[u]$ **do**
 - if** $v \in Q$ **and** $w(u, v) < key[v]$
 - $\pi[v] \leftarrow u$
 - $key[v] \leftarrow w(u, v)$

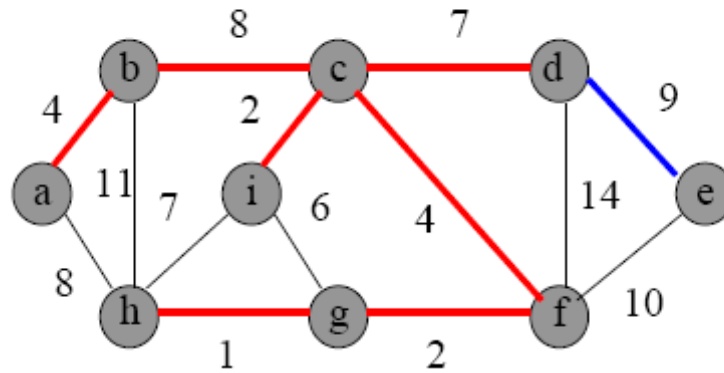
Esempio



(cont.)



(cont.)



Algoritmo di Prim: Analisi

- L'efficienza dell'algoritmo di Prim dipende dalla coda Q
- Se Q viene realizzata tramite uno heap binario:
 - **buildHeap()** per l'inizializzazione: $O(V)$
 - Il ciclo principale viene eseguito $|V|$ volte ed ogni operazione **extractMin()** è $O(\log V)$
 - Il ciclo interno viene eseguito $O(E)$ volte

Come eseguire il controllo di appartenenza ($v \in Q$) in tempo $O(1)$?

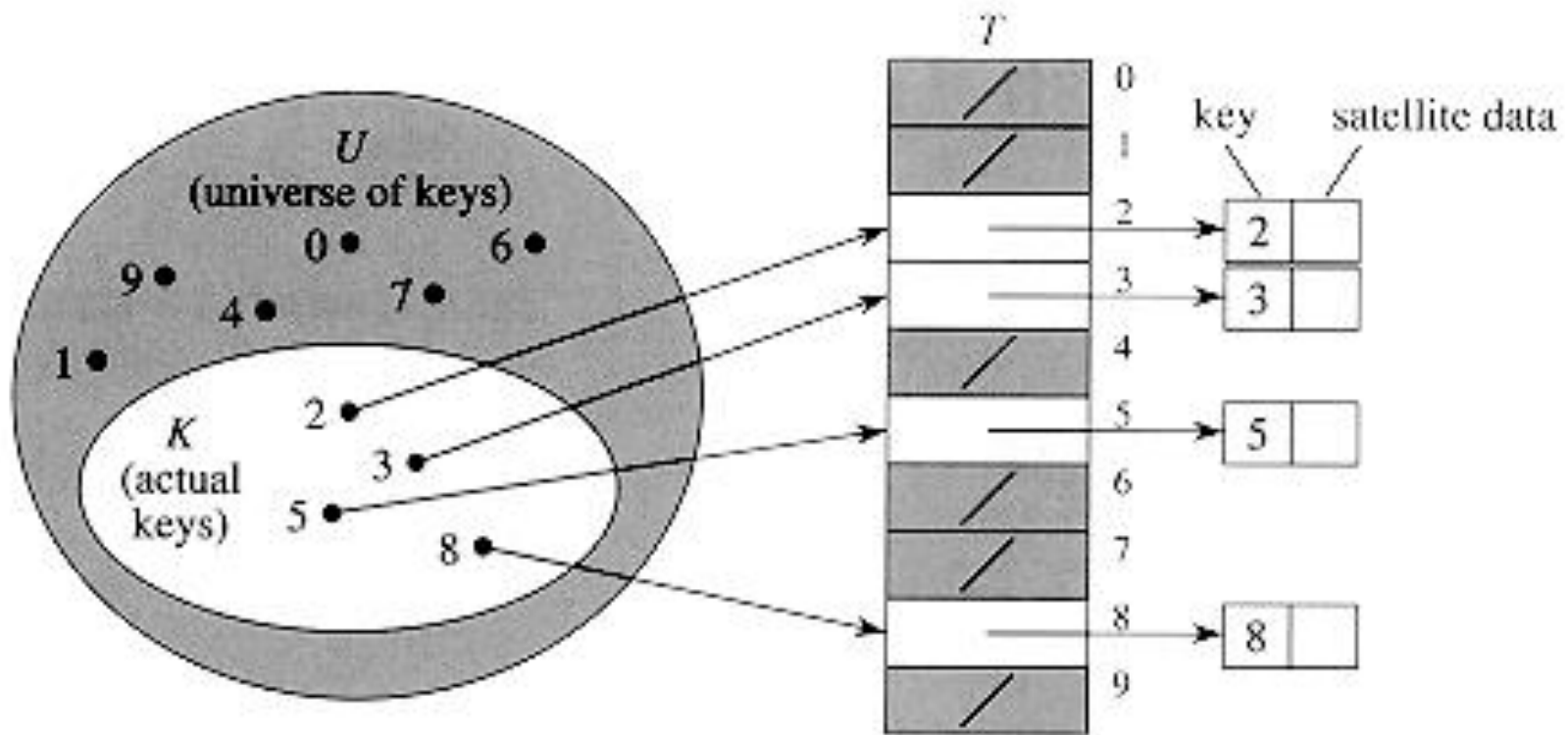
(cont.)

- L'operazione **decreaseKey()** sullo heap che costa $O(\log V)$
- **Tempo totale: $O(V + V \log V + E \log V) = O(E \log V)$ asintoticamente uguale a quello di Kruskal**

Hash Tables

Tabelle di Indirizzamento Diretto

- L'indirizzamento diretto è una tecnica semplice che funziona quando l'universo U delle chiavi è ragionevolmente piccolo.
- Un insieme dinamico in cui ogni elemento ha una chiave ottenuta dall'Universo $U = \{0, \dots, m - 1\}$ con m non troppo grande
- Per rappresentarlo si usa un array o **tabella ad indirizzamento diretto**, che sarà indicata con $T[0, \dots, m - 1]$, in cui in ogni cella corrisponde ad una chiave dell'Universo U .



(cont.)

- Se l'insieme non contiene elementi con la chiave k allora $T[k] = NULL$.
- Le operazioni sulla Tabella, con complessità $O(1)$, sono:
 - $Direct_Address_Search(T, k)$
return $T(k)$
 - $Direct_Address_Insert(T, x)$
 $T[key[x]] \leftarrow x$
 - $Direct_Address_Delete(T, x)$
 $T[key[x]] \leftarrow NULL$

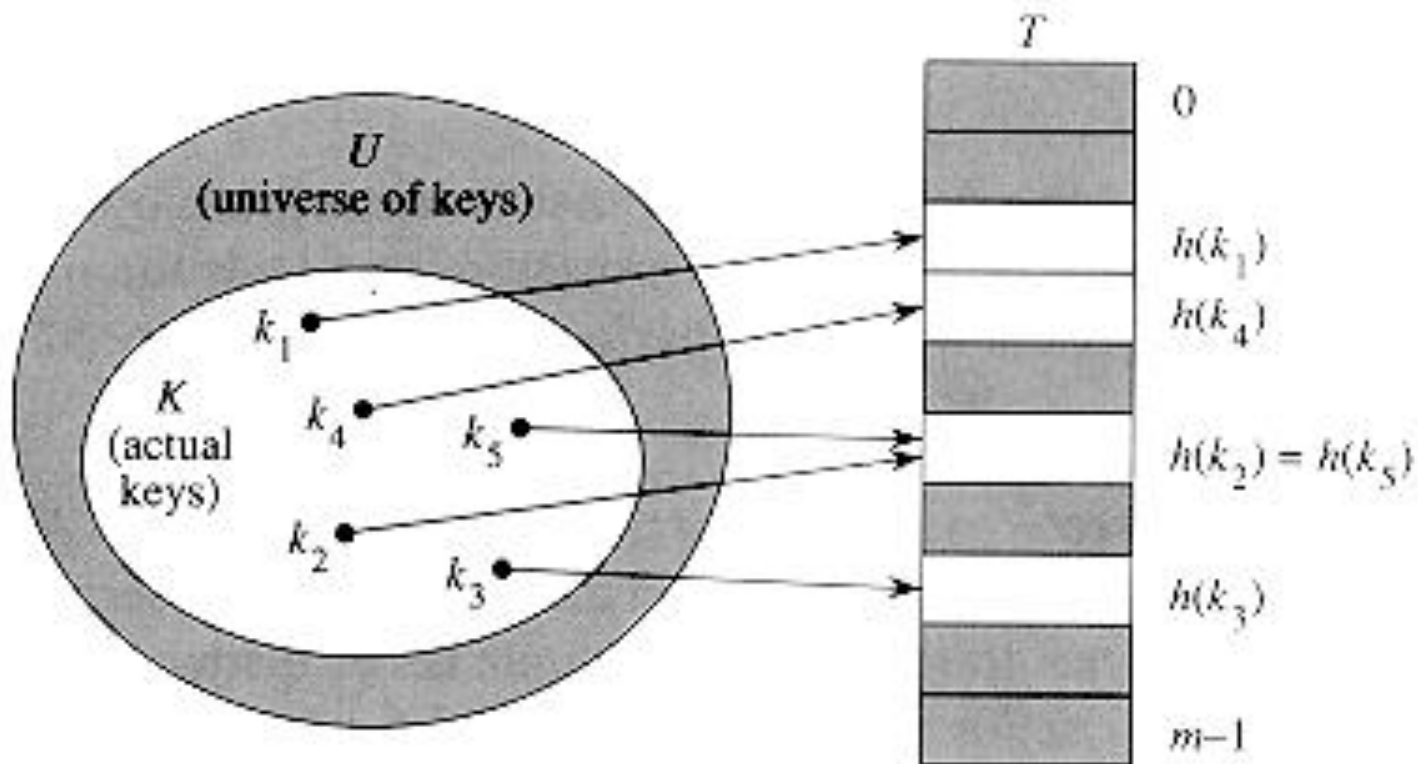
TABELLE HASH: Introduzione

- Se l' universo U è grande, memorizzare una tabella T di dimensione $|U|$ è impraticabile.
- L' insieme K delle chiavi effettivamente utilizzate è molto più piccolo dell' Universo U .
- La Tabella Hash richiede molta meno memoria. La memoria richiesta è pari a $\Theta(|k|)$ e la ricerca richiede ancora solo tempo $O(1)$.
- $O(1)$ vale nel caso medio mentre per le tabelle precedenti è nel caso peggiore.

Tabelle HASH

- Con l'indirizzamento diretto un elemento con chiave K è memorizzato nella posizione K .
- Con le Tabelle Hash viene memorizzato nella posizione $h(K)$ dove h (funzione hash)
 $h: U \rightarrow [0, \dots, m - 1]$. Ossia l'elemento con chiave K viene memorizzato nella posizione $h(K)$. Quindi invece di gestire una tabella con $|U|$ valori se ne gestisce una tabella con $|m|$ valori.

Tabella di Hash



Collisione

- Con il metodo HASH si ha il problema della collisione. Ossia a più chiavi distinte può corrispondere la stessa posizione $h(k)$
- Un' idea è scegliere h in modo da minimizzare le collisioni. Questo può essere realizzato con una funzione hash che sia deterministica ossia per la stessa chiave k deve sempre produrre lo stesso valore $h(k)$ ma che sia “casuale” ossia che il valore $h(k)$ abbia la stessa probabilità di assumere una delle m posizioni della tabella

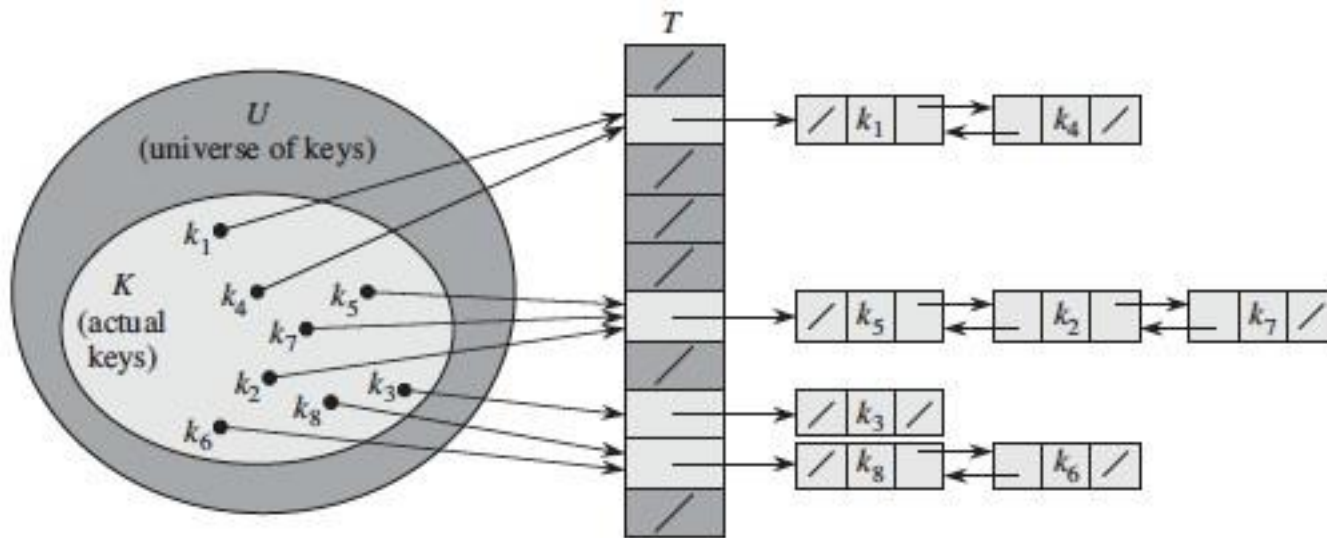
Risoluzione delle collisioni per concatenazione

- Nel metodo della concatenazione si mettono in una lista tutti gli elementi che corrispondono alla stessa posizione $h(k)$; Nella posizione c' è memorizzato un puntatore alla testa della lista. Se il puntatore è NIL non ci sono elementi.

(cont.)

- CHAINED-HASH-INSERT(T, x)
 - **inserisce x in testa alla lista** $T[h(key[x])]$
- CHAINED-HASH-SEARCH(T, k)
 - ricerca un elemento con chiave k nella lista $T[h(k)]$
- CHAINED-HASH-DELETE(T, k)
 - cancella x dalla lista $T[h(key[x])]$.

Risoluzione delle Collisioni: Concatenazione



Complessità Computazionale

- INSERT impiega tempo $O(1)$;
- SEARCH impiega tempo $O(l)$ dove l è la lunghezza della lista;
- DELETE impiega $O(l)$ se le liste sono doppiamente concatenate, in modo da rendere la cancellazione più rapida.

FATTORE DI CARICO

- Data una tabella hash T con m posizioni che memorizza n elementi, si definisce **il fattore di carico** $\alpha = n/m$ ossia il numero medio di elementi per ogni lista memorizzata.
- Si assume che qualunque elemento corrisponda in modo equamente probabile a una delle m posizioni, indipendentemente dalla posizione degli altri elementi (**uniformità semplice della funzione hash**)

Teorema

- In un tabella hash le cui collisioni sono risolte con il concatenamento, una ricerca senza successo richiede un tempo atteso $\Theta(1 + \alpha)$, nell' ipotesi di hashing uniforme.

Dimostrazione

- Nell' ipotesi di hashing uniforme, qualsiasi chiave k non ancora memorizzata nella tavola ha la stessa probabilità di essere associata ad una qualsiasi delle m celle.
- Il tempo atteso per ricercare senza successo una chiave k è il tempo atteso per svolgere le ricerche fino alla fine della lista $T[h(k)]$, che ha lunghezza attesa pari ad α .

Teorema

- In un tavolo hash le cui collisioni sono risolte con il concatenamento, una ricerca con successo richiede un tempo atteso $\Theta(1 + \alpha)$, nell'ipotesi di hashing uniforme.

(cont.)

- Quindi il numero atteso di elementi esaminato in una ricerca senza successo è α e il tempo totale richiesto, incluso quello per calcolare $h(k)$, è $\Theta(1 + \alpha)$.

Dimostrazione

- Supponiamo che l' elemento da ricercare abbia la stessa probabilità di essere uno qualsiasi degli n elementi memorizzati nella tavola.
- Il numero di elementi esaminati durante una ricerca con successo di un elemento x è uno in più del numero di elementi che si trovano prima di x nella lista di x .
- Gli elementi che precedono x nella lista sono stati inseriti tutti dopo di x , perché i nuovi elementi vengono posti all' inizio della lista.

Dimostrazione

- Per trovare il numero atteso di elementi esaminati, prendiamo la media, sugli n elementi x nella tavola, di 1 più il numero atteso di elementi aggiunti alla lista di x , dopo che x è stato aggiunto alla lista.
- Indichiamo con x_i l' i -esimo elemento inserito nella tavola, per $i = 1, \dots, n$ e sia $k_i = \text{key}[x_i]$. Per le chiavi k_i e k_j , definiamo la variabile aleatoria indicatrice $X_{ij} = I\{h(k_i) = h(k_j)\}$.

(cont.)

- Nell' ipotesi di hashing uniforme semplice, abbiamo $Pr\{h(k_i) = h(k_j)\} = \frac{1}{m}$, quindi

$$E[X_{ij}] = \frac{1}{m}.$$

$$\begin{aligned} E\left[\frac{1}{n}\sum_{i=1}^n\left(1 + \sum_{j=i+1}^n X_{ij}\right)\right] &= \frac{1}{n}\sum_{i=1}^n\left(1 + \sum_{j=i+1}^n E[X_{ij}]\right) \\ &= \frac{1}{n}\sum_{i=1}^n\left(1 + \sum_{j=i+1}^n \frac{1}{m}\right) = \frac{1}{n}\sum_{i=1}^n\left(1 + \frac{1}{m}\sum_{j=i+1}^n 1\right) \\ &= \frac{1}{n}\sum_{i=1}^n\left(1 + \frac{1}{m}(n-i)\right) = \frac{1}{n}\sum_{i=1}^n 1 + \frac{1}{nm}\sum_{i=1}^n (n-i) \end{aligned}$$

(cont.)

- Nell' ipotesi di hashing uniforme semplice, abbiamo $Pr\{h(k_i) = h(k_j)\} = \frac{1}{m}$, quindi

$$E[X_{ij}] = \frac{1}{m}.$$

$$E\left[\frac{1}{n} \sum_{i=1}^n \left(1 + \sum_{j=i+1}^n X_{ij}\right)\right] = \frac{1}{n} \sum_{i=1}^n \left(1 + \sum_{j=i+1}^n E[X_{ij}]\right)$$

$$= 1 + \frac{1}{nm} \sum_{i=1}^n (n - i)$$

$$= 1 + \frac{1}{nm} (\sum_{i=1}^n n - \sum_{i=1}^n i) = 1 + \frac{1}{nm} \left(n^2 - n \frac{n+1}{2}\right)$$

$$= 1 + \frac{1}{m} \left(n - \frac{n+1}{2}\right) = 1 + \frac{1}{m} \left(\frac{2n - n - 1}{2}\right) = 1 + \frac{n-1}{2m} = 1 + \frac{1}{2} \frac{n}{m} - \frac{1}{2m} = 1 + \frac{\alpha}{2} - \frac{\alpha}{2n} = \Theta(1) + \Theta\left(\frac{\alpha}{2}\right) = \Theta(1 + \alpha)$$

Funzioni Hash

- Ogni chiave corrisponde a ciascuna delle m posizioni in modo equamente probabile

$$\sum_{k:h(k)=j} P(k) = \frac{1}{m} \quad j = 1, \dots, m$$

Metodo della divisione

- $h(k) = k \bmod m$
- Quando si utilizzano il metodo della divisione, è necessario evitare alcuni valori di m . **È necessario evitare potenze di 2**, infatti se $m = 2^p$ allora m rappresenta i p bit meno significativi. A meno che non sia noto che tutte le configurazioni dei p bit di ordine inferiore abbiano la stessa probabilità, **è meglio rendere la funzione di hash dipendente da tutti i bit della chiave.**

(cont.)

- È scorretto scegliere $m = 2^p - 1$ quando k è una stringa di caratteri interpretata nella base 2^p , perché la permutazione dei suoi caratteri non cambia il suo valore di hash.
- Una buona idea può essere scegliere per m valori primi non troppo vicini a potenze esatte di 2. (ad es. 701)